Design of Intelligent Edge and Time-Series Data Flow Temperature Control System

Разработка интеллектуальной системы управления температурой
на основе периферийных вычислений и потоковой обработки временных рядов

Student / Студент: Wang Gen

Руководитель: В.С. Разумейчик

Department / Кафедра: «ЭВМ и систем»

BrSTU / БрГТУ

2026

Objective And Implemented Scope

Objective

Build an end-to-end prototype for intelligent temperature control.

This work covers system design, software implementation, and prototype validation.

Implemented Scope

1. **Edge-side real-time control**
   Local closed-loop behavior
2. **MQTT-based telemetry flow**
   Continuous device reporting
3. **Time-series data storage**
   Historical query and analysis
4. **HMI monitoring and operation**
   User-facing observation and control
5. **AI-assisted PID tuning**
   Decision support for parameter adjustment

Requirements And Design Goals

Functional Requirements

- Temperature acquisition and control
- Telemetry upload and storage
- Historical query and visualization
- Alarm and parameter management
- AI-assisted recommendation

Design Goals

Safety

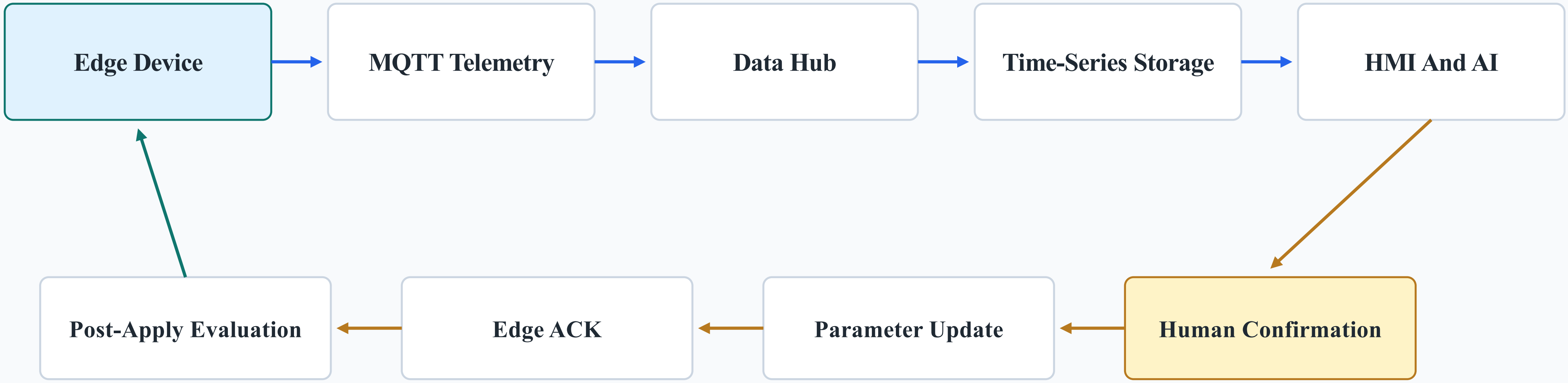
Observability

Traceability

Maintainability

The design focuses on controlled operation, visible runtime behavior, recorded actions, and maintainable system structure.

End-to-End Closed Loop Architecture

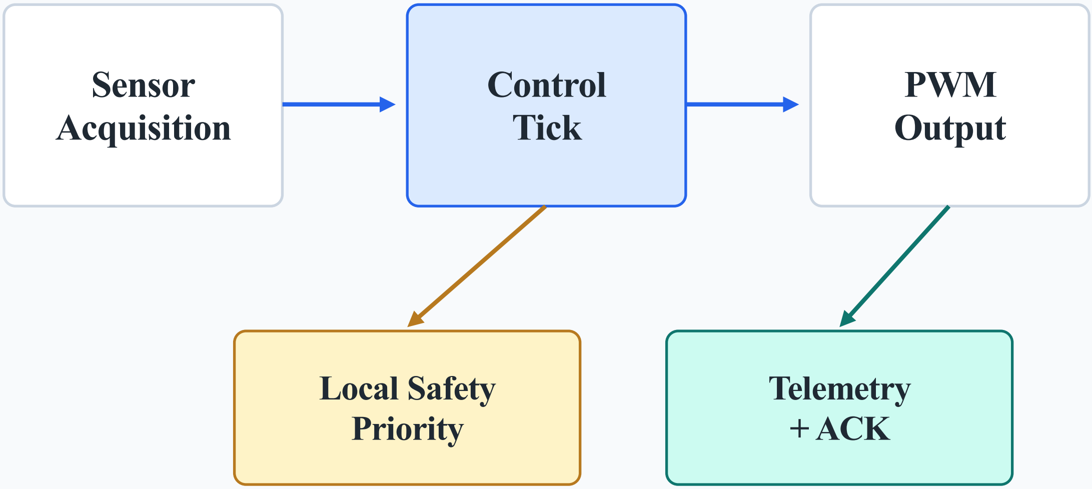


Key Principles

Real-time control remains on the edge.

AI supports PID tuning decisions, not direct actuation.

Edge Control And Hardware Design



Edge Responsibilities

- Sensor acquisition
- Periodic control tick
- PID-compatible parameters
- PWM actuator output
- Output limiting and anti-windup
- Local safety handling
- Parameter ACK

Telemetry And Control Metrics

Telemetry

- Temperature / setpoint / error
- Control output
- PID parameters
- Device status and fault state
- Timestamp and device ID

→ **Control-quality features** →

Control Metrics

- Mean absolute error
- Overshoot
- Settling time
- In-band ratio
- Temperature swing
- Saturation ratio

Telemetry is used for display, control-quality evaluation, and AI feature extraction.

Data Hub Reliability

MQTT → Validation → Filtering → Queue → Aggregation → Storage → Metrics

Processing Responsibilities

- MQTT telemetry ingestion
- JSON parsing and validation
- Abnormal data filtering
- Bounded asynchronous processing

Storage And Observation

- Short-term aggregation
- TDengine time-series storage
- PostgreSQL management data
- Runtime throughput, latency, and failure metrics

Hybrid AI-Assisted PID Tuning

AI Role

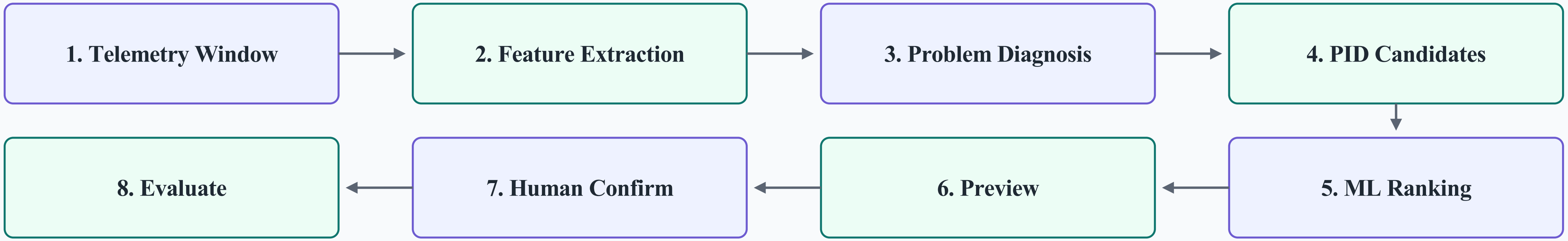
- Decision support for PID tuning
- Not direct temperature control
- Human-confirmed parameter update

Rule-Based Layer

- Telemetry feature extraction
- Control problem diagnosis
- Conservative PID candidate generation

ML Model Layer

- Candidate PID ranking
- Improvement probability estimation
- Preview risk estimation
- Rule fallback if artifacts are unavailable



Models: Logistic Regression / Random Forest   Artifacts: joblib

Deployment And Validation

Deployment

- Edge firmware
- MQTT broker
- Java Data Hub
- TDengine / PostgreSQL
- HMI and AI services

Validation

- End-to-end telemetry flow
- Parameter update with ACK
- Data Hub pressure testing
- Baseline throughput and latency
- Metrics and database maintenance

Results, Limitations And Future Work

Main Results

- End-to-end prototype implemented
- Edge control and telemetry upload verified
- Data Hub ingestion, storage, and metrics verified
- HMI monitoring and parameter management verified
- AI-assisted PID tuning workflow verified

Limitations

- Prototype-level validation
- More physical thermal-load experiments are needed
- Long-term quantitative control evaluation should be extended

Future Work

- Broader device and scenario testing
- Improved adaptive tuning
- Stronger deployment automation and monitoring

The work combines design, implementation, and prototype validation.